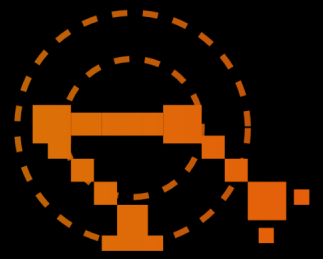
SPRINT 01 PLAN

FELIPE SUASSUNA
LEANDRO GERTRUDES
PEDRO NEIVA
RUI MOURA
VINÍCIUS POLATTO
WALLACE LUCAS



POSITRONIC

PLANNING
19–30 OCT 2026

S01

ARM

THE ARM

What we have, and how we talk to it

03 WHAT WE’RE WORKING WITH
04 HOW WE TALK TO IT

WHAT WE'RE WORKING WITH



myCobot 280 Jetson Nano

UBUNTU 20.04 // L4T R32.6.1 // JETPACK 4.6

6
AXES

280
MM REACH RADIUS

250
G PAYLOAD

4
CPU CORES, A57

4
GB RAM,
SHARED GPU

64
GB STORAGE

HOW WE TALK TO IT

JETSON ⇄ /dev/ttyTHS1 ⇄ ATOM ESP32 // 1 000 000 BAUD, 8N1

ASK →	FE FE	02	2B		FA
	HEADER	LEN	CMD		END
← REPLY	FE FE	03	2B	00/01	FA
	HEADER	LEN	CMD	MOVING?	END

LEN = CMD + DATA + FA // 00 STOPPED, 01 MOVING

PLAN

THE PLAN

What we promise, and how we'll get there

06 SPRINT GOAL
07 GOALS && DONE WHEN
08 THE PATH

SPRINT GOAL

The real arm is visible in ROS 2 from any laptop, and every PR is built and tested by CI.

// Foundations now, so every next sprint is about the arm, not setup

S00 // DONE
Tools, Git, hardware study

S01 // 19–30 OCT
Arm visible in ROS 2

GOALS && DONE WHEN

G1 [A]
C++ communication protocol
Abstract the entire arm byte protocol into a usable C++ library

G2 [A]
/joint_states from the Jetson
The driver node reads the 6 joints and publishes them at measured rate

G3 [B]
Arm in Gazebo and RViz
The URDF builds on Jazzy, the simulated arm runs in Gazebo with ros2_control and the real arm in RViz

G4 [B]
Safe workspace in MoveIt
Table, base and limits as obstacles, so unsafe goals are rejected

G5 [C]
Arm reachable on power on
Turning on the machine is enough to interact with it via ROS 2

G6 [C]
Bullet-proof CI structure
All PRs are checked thoroughly, assuring no broken or nonstandard code ever touches dev/main

THE PATH

ISSUE	WHAT IT GIVES US	GOAL	WHO
#19	A ROS-ready container running on the arm	G5	FELIPE
#20	WCET, so we can set failure timeouts from real data	G2	VINÍCIUS
#21	The C++ library that speaks the arm's byte protocol	G1	WALLACE && LEANDRO
#22	The single door between ROS 2 and the arm	G2	WALLACE && LEANDRO
#26	The capability to simulate our arm in a controlled environment	G3	RUI
#42	Unsafe goals rejected before the arm ever moves	G4	PEDRO

WORK

THE WORK

How we'll deliver it, in practice

10 HOW WE WILL WORK
11 OUR KANBAN
12 GIT IN PRACTICE
13 ROS 2 LINT

HOW WE WILL WORK

FLOW	Issue → branch → PR to dev 2 approvals + green CI → squash
GITHUB	One milestone per sprint main = what we demo, tag v0.N.0
COMMITS	`feat(hardware): read joint angles` Conventional Commits, PR < 400 lines
CODE	ROS 2 Jazzy lint, C++17 clang-tidy: no new warnings

ON THE ARM

One driver at a time

Nobody in its reach

Clear, tidy bench

Slow, small moves

OUR KANBAN



ISSUES Sprint 01 sized with our Claude team plan's time estimates

EVERY CARD: PRIORITY // MODULE // TYPE // SPRINT

GIT IN PRACTICE

GITHUB.COM/SEAME-PT/TEAM3_POSITRONIC

```
// branches: one per issue, created from dev
main                                  // demoed, tagged v0.N.0
dev                                   // default, reviewed work only
21-serial-protocol-library-in-c       // issue #21
26-arm-in-gazebo-with-ros2-control    // issue #26
// commits: type(scope): imperative summary
build(protocol): add positronic_protocol package skeleton
```

EVERY PR: 2 APPROVALS // GREEN CI // SQUASH MERGE

ROS 2 LINT

BEFORE // FAILS

```
using namespace std;
int last(vector<int>& v){
  if(v.empty()) return -1;
  return v.back();
}
```

AFTER // CLEAN

```
int last(std::vector<int> & v)
{
  if (v.empty()) {
    return -1;
  }
  return v.back();
}
```

CPPLINT FLAGS USING NAMESPACE // UNCRUSTIFY FIXES THE REST

Q&A_



POSITRONIC

github.com/SEAME-pt/Team3_Positronic